

MAChip — Technical and Hardware-Software Comprehensive Glossary

Capstone System: Microchip Attendance and Chip Payroll (MAChip)
Client Organization: MAC-J Int'l Forwarding Ltd., Co. (CTPAT-Compliant International Freight Forwarder, NAIA Complex)
Document Classification: Technical Terminology, Hardware-Software Integration, and System Architecture Reference Manual
Target Audience: Academic Panelists, Technical Evaluators, Software Engineers, and Non-Technical Stakeholders

Executive Overview

MAChip (Microchip Attendance and Chip Payroll) is an automated, integrated enterprise hardware-software capstone platform engineered for MAC-J Int'l Forwarding Ltd., Co., a customs-bonded international logistics and air-cargo freight forwarding company located within the Ninoy Aquino International Airport (NAIA) complex.

Because MAC-J operates under strict CTPAT (Customs-Trade Partnership Against Terrorism) guidelines and Philippine Department of Labor and Employment (DOLE) mandates, MAChip bridges the gap between physical perimeter security and digital payroll governance. It unites a dual-core embedded IoT wall terminal (incorporating high-frequency RFID, optical biometric scanning, dual telemetry screens, and electromagnetic lock actuation) with an enterprise-grade web application (Node.js, Express, PostgreSQL, and React).

This document serves as an exhaustive reference manual for all technical terminologies, hardware components, communication protocols, architectural patterns, and cybersecurity mechanisms implemented throughout the system. Every term is broken down into its plain-language definition, real-world operational analogy, technical specifications, concrete MAChip implementation details, and academic defense rationale.

Table of Contents

1. Embedded IoT and Hardware Architecture
2. Software Engineering and Backend Architecture
3. Database Design and Data Integrity Layer
4. Frontend Architecture and User Interface Layer
5. Cybersecurity and Cryptographic Standards
6. Comprehensive Academic Defense Reference Sheet

1. Embedded IoT and Hardware Architecture

ESP32 Dual-Core Microcontroller Architecture

- **Plain-Language Definition:** The primary electronic "brain" residing inside the physical attendance station and door control enclosure. It is a powerful, low-cost system-on-a-chip (SoC) equipped with dual 32-bit computing cores, integrated 2.4 GHz Wi-Fi, Bluetooth, and programmable input/output pins designed to control physical sensors and relay data directly to the local server.
- **Real-World Analogy:** Think of the ESP32 as a vigilant, automated security officer permanently stationed at the warehouse entrance. It checks an employee's badge, reads their fingerprint, validates their identity against local memory, sounds advisory beeps, displays their photo and name on the screen, and sends a radio dispatch to the main office server asking for authorization to release the electric door bolt.
- **Technical Specifications:**
 - Microprocessor: Tensilica Xtensa Dual-Core 32-bit LX6 running at 240 MHz clock frequency.
 - Memory: 520 KB internal SRAM, 4 MB external SPI flash memory.

- **Wireless Connectivity:** 802.11 b/g/n Wi-Fi baseband (operating up to 150 Mbps) and Bluetooth v4.2 BR/EDR and BLE.
 - **Peripherals:** Hardware timers, hardware PWM channels, 3 UART interfaces, 2 SPI interfaces, 2 I2C buses, and 36 programmable General Purpose Input/Output (GPIO) pins.
 - **MAChip System Implementation:** Acts as the master hardware edge controller in `firmware/MaChip_Test/MaChip_Test.ino`. It orchestrates concurrent operations: managing SPI communication with dual MFRC522 RFID readers, driving the ST7789 TFT display, sampling UART packets from the R307S fingerprint sensor, updating the SSD1306 diagnostic OLED over I2C, pulsing the electromagnetic lock relay via GPIO 14, and dispatching encrypted JSON packets over Wi-Fi to the Express backend.
 - **Defense Panel Insight & Rationale:** Panelists frequently ask why an ESP32 was chosen over an Arduino Uno or Raspberry Pi. The answer is twofold: (1) Arduino Uno lacks the multi-megabyte memory and hardware cryptography required for TLS/AES-128 and dual-screen framebuffers; (2) Raspberry Pi represents a full microcomputer running a heavy Linux OS with long boot times and vulnerability to SD card corruption upon abrupt power loss, whereas the ESP32 boots instantly (under 200ms) and executes bare-metal FreeRTOS firmware with zero file-system corruption risk.
 - **Firmware Reference:** `firmware/MaChip_Test/MaChip_Test.ino` (Lines 1 to 65).
-

MFRC522 High-Frequency RFID Module (13.56 MHz Reader)

- **Plain-Language Definition:** A proximity card scanner that emits a high-frequency radio field to read the unique identification number (UID) embedded inside an employee's plastic ID card or keychain fob when held within 3 to 5 centimeters.
 - **Real-World Analogy:** Identical to tapping a contactless Beep card at an MRT/LRT commuter turnstile, or tapping a contactless debit card on a point-of-sale checkout terminal.
 - **Technical Specifications:**
 - Operating Frequency: 13.56 MHz ISM band.
 - Supported Protocols: ISO/IEC 14443 Type A, MIFARE Classic (1K, 4K), MIFARE Ultralight.
 - Communication Interface: Serial Peripheral Interface (SPI) supporting transfer rates up to 10 Mbit/s.
 - Modulation: 100% ASK (Amplitude Shift Keying).
 - **MAChip System Implementation:** Configured in a **Dual-Reader Architecture** sharing the primary hardware SPI bus (MOSI=23, MISO=19, SCK=18), but separated by independent Slave Select (SS) lines. The IN-Reader (`SS_PIN_IN = 5` , `RST_PIN_IN = 22`) is mounted at the exterior doorway to register clock-in events; the OUT-Reader (`SS_PIN_OUT = 26` , `RST_PIN_OUT = 21`) is mounted on the interior wall to register clock-out events. The firmware continuously alternates slave select polling every 50 milliseconds.
 - **Defense Panel Insight & Rationale:** When asked how the system prevents signal collisions between two RFID readers sharing the same SPI lines, explain that SPI uses bus multiplexing via dedicated Chip Select (CS/SS) lines. When the ESP32 pulls Pin 5 LOW, the IN-reader is active and the OUT-reader's MISO pin enters high-impedance state (tristate); when Pin 26 is pulled LOW, the roles reverse.
 - **Firmware Reference:** `firmware/MaChip_Test/MaChip_Test.ino` (Lines 26 to 30, 85 to 110).
-

Optical Fingerprint Sensor (R307S) and Biometric Template Matching

- **Plain-Language Definition:** A high-precision scanner that projects light onto an employee's finger, captures an optical image of the microscopic ridges and valleys of the skin, converts that image into a mathematical digital template (a sequence of encrypted numbers, never a raw image photo), and compares it against enrolled templates stored in its secure internal memory.
- **Real-World Analogy:** Exactly like the biometric fingerprint sensor found on smartphones or automated banking teller machines.
- **Technical Specifications:**
 - Optical Resolution: 500 DPI.
 - Sensor Window: 19 mm x 21 mm optical glass prism.

- Internal Processing: High-speed digital signal processor (DSP) capable of template extraction and 1:N matching in under 0.8 seconds.
 - Communication Interface: Asynchronous UART TTL Serial (Baud Rate: 57,600 bps).
 - Biometric Capacity: Up to 1,000 distinct fingerprint character templates.
 - **MaChip System Implementation:** Connected to the ESP32 via Hardware Serial 2 (`FP_RX = 16` , `FP_TX = 17`). Acts as the non-repudiable second factor of authentication (2FA). Following an RFID card tap, the system requires the employee to place their finger on the R307S sensor within a 15-second window (`TIMEOUT_2FA = 15000`). If verified, the template ID is cross-referenced with the cardholder's record.
 - **Defense Panel Insight & Rationale:** Why is biometrics paired with RFID? RFID cards can easily be handed to a coworker to fraudulently record attendance for an absent or tardy employee (a widespread logistics industry issue known as "buddy punching"). Biometrics ensures physical presence. Storing mathematical minutiae templates rather than raw image files also upholds Philippine Data Privacy Act (RA 10173) compliance, as minutiae templates cannot be reverse-engineered back into a finger photo.
 - **Firmware Reference:** `firmware/MaChip_Test/MaChip_Test.ino` (Lines 22 to 23, 140 to 195).
-

Solenoid Electronic Door Deadbolt and Optocoupler Relay

- **Plain-Language Definition:** A heavy-duty electromagnetic locking bolt that physically locks or unlocks the facility entrance door. An electronic relay acts as an isolated switch that allows the low-voltage ESP32 (3.3V) to safely control the high-power 12V direct current powering the magnetic lock.
 - **Real-World Analogy:** Like the electronic door buzzer in a high-security bank entrance, jewelry shop, or embassy gate that makes a loud click when security presses a button under the counter, releasing the door for a few seconds.
 - **Technical Specifications:**
 - Operating Voltage: 12V DC.
 - Peak Current Draw: 1.1A to 1.5A during solenoid energization.
 - Relay Module: 5V Single-Channel Relay with optical optocoupler isolation.
 - Fail-Safe Configuration: Fail-Secure (remains locked during power interruptions, requiring manual physical master key override for safety).
 - Protection: 1N4007 flyback diode placed across solenoid terminals to absorb inductive reverse-voltage spikes.
 - **MaChip System Implementation:** Driven by ESP32 Pin 14 (`SOLENOID_PIN = 14`). When both RFID and Fingerprint criteria are validated by the backend, the ESP32 triggers the relay for exactly 3,000 milliseconds (`SOLENOID_DURATION = 3000`). This unlatches the deadbolt, allowing the employee to push the warehouse door open, after which the relay automatically de-energizes and the spring bolt re-locks.
 - **Defense Panel Insight & Rationale:** Why is an optocoupler relay mandatory instead of connecting the solenoid directly to a GPIO pin? An ESP32 GPIO pin can supply a maximum of 12 milliamperes at 3.3V. The solenoid requires 1,200 milliamperes at 12V. Furthermore, when an electromagnet turns off, its collapsing magnetic field generates a massive reverse voltage surge (inductive kickback of over 50V) that would instantly destroy the microcontroller. The optocoupler uses an internal light-emitting diode and phototransistor to isolate the two circuits completely with zero physical electrical contact.
 - **Firmware Reference:** `firmware/MaChip_Test/MaChip_Test.ino` (Lines 20, 46 to 47, 210 to 225).
-

Dual-Display Architecture (ST7789 Color TFT and SSD1306 Monochrome OLED)

- **Plain-Language Definition:** Two physical display screens connected to the terminal enclosure: a large 2.4-inch full-color screen positioned on the front face for employees, and a compact 0.96-inch monochrome screen positioned on the rear side for IT technicians and facility managers.
- **Real-World Analogy:** Exactly like a point-of-sale cash register at a supermarket. The customer looks at a colorful front screen showing their scanned items, total bill, and payment confirmation, while the cashier looks at a smaller technical monitor displaying system diagnostics, register numbers, and network status.
- **Technical Specifications:**

- Front Display: ST7789 IPS Color TFT, 240x240 pixel resolution, 65K full-color RGB565 palette, driven over high-speed hardware SPI.
- Rear Display: SSD1306 Monochrome OLED, 128x64 pixel resolution, ultra-low power emissive display, driven over I2C (SDA=Pin 4, SCL=Pin 15).
- **MAChip System Implementation:**
 - Front ST7789: Renders greeting banners, employee full names, employee ID numbers, time-in/time-out timestamps, and color-coded status badges (Green = On-Time, Orange = Grace Period, Red = Late / Access Denied).
 - Rear SSD1306: Renders system telemetry including the local IP address assigned by the DHCP router, server ping latency in milliseconds, Wi-Fi Received Signal Strength Indicator (RSSI in dBm), and NTP real-time clock synchronization state.
- **Defense Panel Insight & Rationale:** Dual screens separate user-experience concerns from maintenance concerns. Employees receive clean, immediate visual confirmation without clutter, while facility technicians can diagnose network disconnects or IP address changes instantly without opening the physical enclosure or connecting a diagnostic laptop.
- **Firmware Reference:** `firmware/MaChip_Test/MaChip_Test.ino` (Lines 31 to 42, 230 to 310).

Hardware Communication Protocols (SPI, I2C, and UART)

- **Plain-Language Definition:** The three standardized digital electronic "languages" and wire configurations that microchips use to transmit data back and forth across a circuit board.
- **Real-World Analogy:**
 - SPI is like a dedicated express multi-lane highway reserved for bullet trains moving large volumes of data at high speed.
 - I2C is like a shared residential street where multiple slow-moving delivery vans take turns using designated house numbers (addresses).
 - UART is like a private telephone walkie-talkie connecting two guards in a direct, two-wire conversation.
- **Technical Comparison Matrix:**

Protocol	Full Name	Wire Count	Max Speed	Bus Type	MAChip Device Assignment
SPI	Serial Peripheral Interface	4 wires (MOSI, MISO, SCK, CS)	Up to 40 MHz	Synchronous Master-Slave	Dual MFRC522 Readers, ST7789 Color TFT
I2C	Inter-Integrated Circuit	2 wires (SDA, SCL)	Up to 400 kHz	Synchronous Multi-Master Bus	SSD1306 Diagnostic OLED
UART	Universal Asynchronous Receiver-Transmitter	2 wires (TX, RX)	Up to 115.2 kbps	Asynchronous Point-to-Point	R307S Optical Fingerprint Scanner

- **MAChip System Implementation:** The ESP32's hardware multiplexer assigns SPI to high-bandwidth display frames and rapid RFID polling, I2C to the low-bandwidth status OLED, and hardware UART 2 to the fingerprint sensor, ensuring that sensor operations do not bottleneck one another.
- **Firmware Reference:** `firmware/MaChip_Test/MaChip_Test.ino` (Bus initialization routines).

Over-The-Air (OTA) Remote Firmware Updating

- **Plain-Language Definition:** The capability to compile and upload updated software directly into the ESP32 microcontroller wirelessly over the local Wi-Fi network, eliminating the need to physically unscrew the wall unit or plug in a USB programming cable.
- **Real-World Analogy:** Identical to an over-the-air smartphone operating system update (such as an iOS or Android update) that downloads and installs over your home Wi-Fi while keeping all physical components untouched.
- **Technical Specifications:** Implemented using the `ArduinoOTA` framework paired with Multicast DNS (`ESPmDNS`), publishing the terminal service hostname as `machip-esp32.local`.
- **MAChip System Implementation:** When maintenance or timing adjustments are required (such as modifying relay dwell times or screen color schemes), the engineering team compiles the firmware in Arduino IDE or PlatformIO and uploads the

binary image directly to the terminal's IP address. Internal flash memory is partitioned with dual OTA slots (OTA0 and OTA1) for safe fallback if an update fails mid-transmission.

- **Defense Panel Insight & Rationale:** In an industrial logistics warehouse, wall terminals are mounted on physical doorframes inside sealed plastic enclosures. Having to unscrew the casing, unplug the terminal, and attach a micro-USB cable for every code tweak introduces unacceptable operational downtime. OTA allows updates to be deployed in 20 seconds over the LAN.
 - **Firmware Reference:** [firmware/MaChip_Test/MaChip_Test.ino](#) (Lines 7 to 8, 75 to 84).
-

Active Piezoelectric Buzzer with PWM Audio Feedback

- **Plain-Language Definition:** A small audio transducer that generates audible beeps and alerts by vibrating a piezoelectric ceramic disc using electronic frequencies (Pulse Width Modulation).
 - **Real-World Analogy:** The distinct checkout sounds at a grocery scanner—a cheerful high-pitched double-beep when a barcode is successfully scanned, compared to a harsh, low buzz when a coupon or payment card is rejected.
 - **MaChip System Implementation:** Controlled via GPIO 27 (`BUZZER = 27`). The firmware executes distinct sound patterns:
 - Success Affirmation (Access Granted): Two short 2,400 Hz chirps (100ms on, 50ms off, 100ms on).
 - Biometric Prompt (Awaiting Fingerprint): Single rising tone (1,800 Hz for 150ms).
 - Denial Alert (Unauthorized / Tardy / Tamper): Long low-frequency buzz (450 Hz for 800ms).
 - System Warning (Network Disconnected): Three rapid warning pulses (1,000 Hz).
 - **Firmware Reference:** [firmware/MaChip_Test/MaChip_Test.ino](#) (Lines 24, 125 to 135).
-

2. Software Engineering and Backend Architecture

RESTful API Architecture

- **Plain-Language Definition:** A standardized set of communication rules and web link conventions that allows the frontend web browser to request information from or submit data to the backend database server using standard internet verbs.
 - **Real-World Analogy:** A restaurant waiter. You (the client browser) sit at the table with the menu (the user interface). You tell the waiter what dish you want (the API request). The waiter walks back to the kitchen (the backend server and database), and returns carrying your prepared dish (the JSON response).
 - **Technical Specifications:**
 - Protocol: Hypertext Transfer Protocol (HTTP/1.1) over TCP/IP.
 - Data Format: JSON (JavaScript Object Notation).
 - Core Verbs: `GET` (retrieve records), `POST` (create new records), `PUT` / `PATCH` (update existing records), `DELETE` (remove or soft-delete records).
 - Architecture: Stateless communication (every request contains complete authentication context).
 - **MaChip System Implementation:** Structured in modular route files within `backend/src/routes/`:
 - `user.routes.js` : User registration, profile updates, and role assignments.
 - `attendance.routes.js` : Biometric punches, manual time adjustments, and daily logging reports.
 - `userRequest.routes.js` : Overtime filings, vacation/sick leave approvals, and schedule adjustments.
 - `payroll.routes.js` : Batch payroll calculations, payslip generation, and deduction adjustments.
 - **Code Reference:** [backend/src/routes/](#) and [backend/src/controllers/](#).
-

JWT (JSON Web Token) Authentication

- **Plain-Language Definition:** A compact, digitally signed electronic security badge issued by the server when an employee logs in with their credentials. Once issued, the client browser presents this digital badge with every subsequent request to

prove who they are without having to re-type their password.

- **Real-World Analogy:** A stamped, tamper-proof wristband given to you at a private concert or VIP club. Once security inspects your government ID at the main entrance, you merely show your wristband to order refreshments or enter VIP zones, rather than pulling out your ID card every thirty seconds.
 - **Technical Specifications:**
 - Standard: RFC 7519.
 - Structure: Three Base64Url-encoded sections separated by dots: `Header.Payload.Signature`.
 - Cryptographic Signing: HMAC-SHA256 using a server-side 256-bit secret key (`process.env.JWT_SECRET`).
 - Claims Stored: `user_Id` , `user_Email` , `role_Id` , `issuedAt` (`iat`), and `expiration` (`exp`).
 - **MAChip System Implementation:** Generated inside `auth.controller.js` upon credential validation. Attached to outgoing server responses inside a secured, browser-sandboxed cookie (`machip_token`). The `auth.js` middleware parses and verifies this token on every protected route. If the token signature is forged or the token has expired, the request is instantly rejected with an HTTP 401 Unauthorized status.
 - **Code Reference:** `backend/src/middleware/auth.js` (Lines 1 to 44) and `backend/src/controllers/auth.controller.js`.
-

HttpOnly and SameSite Cookie Security Policy

- **Plain-Language Definition:** A specialized browser configuration for storing authentication passes. "HttpOnly" instructs the browser that JavaScript code running inside the web page is strictly forbidden from reading or copying the token. "SameSite=Lax" instructs the browser to never send the token when external third-party websites initiate requests.
 - **Real-World Analogy:** Storing valuable cash inside a welded steel vault bolted to the floor of a bank, rather than keeping loose banknotes in an open coat pocket. A pickpocket who brushes past you on the street cannot reach inside the bolted steel safe.
 - **Technical Specifications:** Set via HTTP response header: `Set-Cookie: machip_token=<JWT>; HttpOnly; SameSite=Lax; Path=/; Max-Age=86400`.
 - **MAChip System Implementation:** Completely mitigates **Cross-Site Scripting (XSS)** token exfiltration and **Cross-Site Request Forgery (CSRF)** session hijacking. Even if an attacker manages to inject a malicious script into a comment or profile field, the script cannot execute `document.cookie` to steal the user's active session.
 - **Code Reference:** `backend/src/controllers/auth.controller.js` (Lines 50 to 55).
-

Role-Based Access Control (RBAC) Hierarchy

- **Plain-Language Definition:** A strict operational access policy that grants system capabilities and displays interface menus exclusively based on an employee's verified job role within the logistics firm.
- **Real-World Analogy:** A commercial office keycard system. Regular warehouse workers can unlock the entrance door and cafeteria; departmental managers can unlock team conference rooms and approve timecards; accounting personnel can access the payroll safe; and only the general director holds the master key that opens all facilities.
- **Technical Specifications:** Enforced through middleware functions (`requireMaster` , `requireAdmin` , `requireOps` , `requireAccountant`) that evaluate `req.user.role_Id` before permitting access to controller execution paths.
- **MAChip Four-Tier Role Matrix:**

Role ID	Role Title	System Capabilities and Operational Scope
Role 1	Admin Manager	Master administrator with unrestricted privileges: user account creation, master reference tables, audit log inspection, and global system configurations.
Role 2	Supervisor	Operational oversight: reviews and approves departmental overtime requests, vacation leaves, sick leaves, and verifies attendance logging reports.
Role 3	Employee	Self-service portal: views personal attendance history, inspects historical payslips, and files leave or overtime requests. Strictly blocked from coworker data.
Role 4	Admin Accountant	Financial management: executes batch payroll generation, adjusts non-taxable allowances, manages loan ledgers, and generates DOLE compliance reports.

- **Code Reference:** `backend/src/middleware/roleCheck.js` (Lines 1 to 65).

Multi-Tier Rate Limiting and Denial-of-Service (DoS) Throttling

- **Plain-Language Definition:** An automated network gatekeeper that counts how many requests a single computer or IP address submits to the server within a specified time window, temporarily blocking clients that exceed safety thresholds.
- **Real-World Analogy:** An automated banking ATM that locks your account card if an incorrect PIN is entered three times in a row, preventing a thief from systematically guessing every four-digit combination.
- **Technical Specifications:** Implemented using `express-rate-limit` using a sliding-window memory counter:
- `loginLimiter` : Maximum of 15 login attempts per 15-minute window per IP address. Exceeding this triggers an HTTP 429 Too Many Requests response.
- `generalLimiter` : Caps general API traffic to prevent resource exhaustion from automated scrapers while permitting legitimate polling.
- **MACHip System Implementation:** Protects the on-premise Mini PC server from brute-force password attacks and Denial of Service (DoS) conditions, ensuring high availability during peak morning clock-in rushes.
- **Code Reference:** `backend/src/middleware/rateLimiter.js` (Lines 1 to 60).

WebSockets and Event-Driven Architecture (Socket.io)

- **Plain-Language Definition:** An open, bidirectional communication channel between the client's web browser and the backend server that stays permanently connected, allowing the server to push instant updates to the browser without requiring the user to refresh the page.
- **Real-World Analogy:** Standard web requests (HTTP) are like sending letters in the postal mail and waiting days for a written response. WebSockets is like an open telephone call where both parties can speak the split-second an event occurs.
- **MACHip System Implementation:** Configured in `backend/src/config/socket.js` . When an employee successfully clocks in at the physical door terminal, the backend receives the hardware punch, updates PostgreSQL, and immediately emits an `attendance_update` event over Socket.io. The Admin Attendance Dashboard catches this broadcast instantly, updating the employee's status badge to green without manual page reloads.
- **Code Reference:** `backend/src/config/socket.js` (Lines 1 to 35).

3. Database Design and Data Integrity Layer

PostgreSQL Relational Database Management System (RDBMS)

- **Plain-Language Definition:** An enterprise-grade, highly reliable open-source database engine that organizes corporate information into structured tables with fixed columns and relational keys.

- **Real-World Analogy:** An organized company filing room. Each employee has an official master file folder tagged with an employee ID number. Their daily attendance cards, monthly payroll slips, and leave application forms are filed in separate drawers, each cross-referenced with that same employee ID number.
 - **Technical Specifications:**
 - Compliance: Full ACID (Atomicity, Consistency, Isolation, Durability) compliance.
 - Data Types: Serial integer primary keys, `double precision` for monetary calculations and accumulated work hours, `smallint` for lookup IDs, and ISO 8601 timestamps.
 - Concurrency: Multi-Version Concurrency Control (MVCC) enabling simultaneous reads and writes without table-level locking.
 - **MAChip System Implementation:** Stores all core business entities across five distinct modules: User Management, User Logging, User Request, Payroll, and System Settings.
 - **Configuration Reference:** `backend/src/config/db.config.js` and `backend/src/config/sequelize.js`.
-

Sequelize Paranoid Mode (Soft Deletion Framework)

- **Plain-Language Definition:** A database protection mechanism where clicking "Delete" on an employee or record **never actually erases or destroys the row** from the storage drive. Instead, the database automatically records a timestamp in a `deletedAt` column, hiding the record from everyday views while permanently preserving its historical relationships.
 - **Real-World Analogy:** Moving an old business document into an archived, locked storage room or placing a computer file into the "Recycle Bin" rather than feeding it into a paper shredder.
 - **Technical Specifications:** Configured with `paranoid: true` on the Sequelize model definition. Normal queries automatically append `WHERE "deletedAt" IS NULL`. Queries requiring audit records can explicitly include soft-deleted rows using `paranoid: false`.
 - **MAChip System Implementation:** Mandated by **CTPAT logistics compliance and DOLE labor standards**. When an employee resigns or is terminated, their profile is soft-deleted to revoke door access and login permissions. However, their 5-year historical payroll calculations, tax remittances, and door-access logs remain intact for legal compliance audits. Hard deletion is strictly blocked if dependent records exist in `Payroll`, `user_logging`, or `emp_Request`.
 - **Code Reference:** `backend/src/config/sequelize.js` and `backend/src/controllers/user.controller.js` (Lines 1300 to 1350).
-

Parameterized SQL Replacements (`:param` Named Placeholders)

- **Plain-Language Definition:** A secure technique for writing database commands where user-provided inputs are treated strictly as harmless text data, completely separated from the executable database instructions.
 - **Real-World Analogy:** Slipping a sealed letter through a narrow mail slot into a secure bank counter, where the teller reads the text on the paper, rather than allowing an unvetted stranger to walk behind the counter and write instructions directly on the master accounting chalkboard.
 - **Technical Specifications:** Every query executed via `sequelize.query()` passes dynamic variables through the `replacements: { paramName }` configuration dictionary rather than concatenating JavaScript string templates.
 - **Vulnerability Prevention:** Completely eradicates **SQL Injection (SQLi)** vulnerabilities. An attacker submitting malicious SQL syntax (such as `' OR '1'='1'; DROP TABLE "User"; --`) has their input sanitized and evaluated as a harmless search string, neutralizing the attack.
 - **Code Pattern:** `javascript // SECURE PATTERN: Enforced across all MAChip controllers await sequelize.query( `SELECT * FROM "User" WHERE "user_Id" = :userId AND "deletedAt" IS NULL`, { replacements: { userId }, type: QueryTypes.SELECT } );`
-

Database Transactions and ACID Guarantees (`BEGIN` , `COMMIT` , `ROLLBACK`)

- **Plain-Language Definition:** An "all-or-nothing" execution safeguard for complex, multi-step operations. If all steps in a sequence succeed, the changes are permanently saved (`COMMIT`). If even one step encounters an error, the database

instantly rewinds all preceding steps to the original state (`ROLLBACK`), preventing corrupted, half-finished records.

- **Real-World Analogy:** A bank fund transfer. The bank must (1) deduct \$500 from Account A, and (2) deposit \$500 into Account B. If the power cuts out after Step 1 but before Step 2, the bank rewinds Step 1 so Account A's money is not permanently lost into thin air.
 - **MACHip System Implementation:** Utilized during annual leave balance initialization, batch user registrations, and batch payroll generation. In `leaveBalanceHelper.js`, credits for all employees are initialized within a managed transaction. If any database constraint fails during loop iteration 25, the entire transaction is rolled back cleanly.
 - **Code Reference:** `backend/src/utils/leaveBalanceHelper.js` (Lines 14 to 65).
-

Immutable Historical Payroll Snapshots (Audit Lock)

- **Plain-Language Definition:** The architectural practice of permanently freezing financial wage values at the exact moment a transaction is executed, ensuring that future salary increases or position updates never alter past accounting records.
 - **Real-World Analogy:** A printed supermarket receipt. When you purchase groceries on June 1st for ₱500, your receipt says ₱500 forever. Even if the supermarket raises the price of those goods to ₱650 in July, your past June receipt does not change.
 - **MACHip System Implementation:** In the database architecture, `User.dailyRate` is the employee's live, editable compensation figure. However, `Payroll.dailyRate` is an **immutable historical snapshot** locked at the exact millisecond payroll is generated. If an employee receives a salary raise from ₱700 to ₱850 in November, their past January through October payslips remain permanently locked at ₱700 for tax and DOLE audit integrity.
 - **Code Reference:** `backend/src/controllers/payroll.controller.js` (Lines 990 to 1015).
-

4. Frontend Architecture and User Interface Layer

React and Single Page Application (SPA) Architecture

- **Plain-Language Definition:** A modern web application design where clicking navigation buttons or submitting forms updates specific parts of the screen dynamically without the web browser flashing white or reloading the entire page from scratch.
 - **Real-World Analogy:** Using an interactive smartphone application where pages slide smoothly and respond instantly, rather than browsing an old-fashioned 1990s web portal that flickers and reloads on every single mouse click.
 - **Technical Specifications:**
 - Build Tool: Vite 5.x for ultra-fast Hot Module Replacement (HMR) and optimized production bundling.
 - Rendering Engine: React 18 with Virtual DOM reconciliation.
 - Routing: Client-side routing via React Router DOM.
 - **MACHip System Implementation:** Powers the responsive administration console, allowing HR managers to toggle between real-time Attendance Logs, Request Approval Queues, and Batch Payroll Processing tabs with zero page reload latency.
 - **Code Reference:** `frontend/src/App.jsx` and `frontend/src/pages/`.
-

Client-Side Storage Hierarchy

- **Plain-Language Definition:** The distinct memory tiers available within a web browser, categorized by how long data persists and who can access it.
- **Technical Comparison Matrix:**

Storage Tier	Persistence Lifetime	Scope	Security Posture	MAChip Assignment
React State	Temporary (resets on component unmount or page refresh)	Component-level	In-memory only; immune to external persistence theft	Form inputs, modal visibility, active tab index
SessionStorage	Lasts only as long as that specific browser tab remains open	Single browser tab	Cleared when the tab is closed	Temporary multi-step payroll review filters
LocalStorage	Permanent (survives browser restarts and system reboots)	Entire origin	Accessible to all client-side JavaScript; vulnerable to XSS	Non-sensitive UI preferences (sidebar toggle, display mode)
HttpOnly Cookie	Configurable (e.g., 24 hours via server expiration)	Browser network engine	Inaccessible to JavaScript; highest security posture	Active authentication session token (<code>machip_token</code>)

- **Security Rationale:** Critical credentials like JWTs are strictly prohibited from `localStorage` in MAChip to prevent token theft via Cross-Site Scripting (XSS).
- **Code Reference:** `frontend/src/services/api.js`.

N+1 Query Waterfall Resolution and Server-Side Batching

- **Plain-Language Definition:** A severe software performance bottleneck where a system requests a list of items (1 request), and then executes an additional separate request for every individual item on the list (N requests), multiplying network delays.
- **Real-World Analogy:** A waiter serving a banquet table of 30 guests by walking back to the kitchen 30 separate times to carry one plate at a time, taking 30 minutes, instead of placing all 30 plates onto a rolling banquet cart and serving the entire table in a single 30-second trip.
- **MAChip Architectural Resolution:**
 - Previous Bottleneck: The payroll preview loop executed sequential HTTP fetches for 30 employees (`for (const emp of employees) { await fetch(...)}`), causing the draft payroll screen to hang for ~30 seconds.
 - Implemented Optimization: Built a dedicated server-side batch endpoint (`GET /api/payroll/preview-batch`). The backend fetches all active employees, parses time logs concurrently in chunks of 6 (`chunkSize = 6`), and returns the entire payroll ledger in a single JSON payload.
 - Result: Load times dropped from **~30,000 milliseconds down to 171 milliseconds (a 175x speedup)**.
 - **Code Reference:** `backend/src/controllers/payroll.controller.js` (Lines 935 to 1025) and `frontend/src/pages/admin_Payroll/PayrollPeriod.jsx`.

5. Cybersecurity and Cryptographic Standards

AES-256-CBC vs. AES-128-CBC Symmetric Encryption

- **Plain-Language Definition:** The Advanced Encryption Standard (AES)—the internationally recognized cryptographic standard trusted by defense agencies and financial institutions. It transforms readable text into unreadable ciphertext using a secret key.
- **Real-World Analogy:** An industrial bank vault with billions of mechanical tumbler combinations. Without the exact physical key, brute-forcing the combination would take billions of years using the world's most advanced supercomputers.
- **Technical Comparison Matrix:**

Parameter	AES-256-CBC	AES-128-CBC
Key Length	256 bits (32 bytes)	128 bits (16 bytes)
Block Size	128 bits (16 bytes)	128 bits (16 bytes)
Initialization Vector (IV)	16 bytes (cryptographically random per record)	16 bytes (generated per transmission)
Execution Environment	Node.js Server (where computing power is abundant)	ESP32 Microcontroller (optimized for memory/battery)
MACHip Application	Database field encryption for employee bank accounts	Wireless transit encryption for biometric punch packets

- **Code Reference:** `backend/src/utils/encryption.js` and `backend/src/middleware/espValidator.js`.

HMAC-SHA256 Cryptographic Message Authentication

- **Plain-Language Definition:** A cryptographic digital signature that guarantees a message has not been altered, modified, or forged while traveling across the computer network.
- **Real-World Analogy:** A tamper-evident holographic security seal stamped over an envelope flap. If a third party intercepts the envelope and tampers with the letter inside, the seal shatters, and the recipient immediately detects the tampering.
- **MACHip System Implementation:** The ESP32 calculates a SHA-256 hash of its JSON payload combined with a pre-shared secret key, sending the resulting hex digest in the request header. The Node.js `espValidator.js` middleware independently calculates the signature. If a single byte was altered in transit, the calculated signature will not match, and the request is rejected with an HTTP 403 Forbidden status.
- **Code Reference:** `backend/src/middleware/espValidator.js` (Lines 59 to 80).

Anti-Replay Protection and Timestamp Skew Validation

- **Plain-Language Definition:** A security validation requiring every hardware transmission to include the exact current second. If the server receives a transmission whose timestamp differs from the server clock by more than 120 seconds, it is rejected as expired.
- **Real-World Analogy:** An electronic movie ticket valid only between 8:00 PM and 8:02 PM. Even if someone records or photocopies your ticket, they cannot present it tomorrow because the timestamp has expired.
- **Vulnerability Mitigated:** Stops **Replay Attacks**, where a malicious actor uses network sniffing tools to capture the valid radio transmission of an authorized manager unlocking the door at 1:00 PM, and replays that recorded signal at 2:00 AM to unlock the warehouse door.
- **Code Reference:** `backend/src/middleware/espValidator.js` (Lines 41 to 54).

CTPAT (Customs-Trade Partnership Against Terrorism) Supply Chain Security

- **Plain-Language Definition:** A comprehensive international security certification led by U.S. Customs and Border Protection for international freight forwarders, air-cargo handlers, and customs-bonded logistics providers.
- **Real-World Analogy:** The gold-standard security clearance that permits international cargo planes and freight trucks to move through international customs checkpoints on an expedited green-lane without being subjected to multi-day physical cargo inspections.
- **MACHip System Fulfillment:** MAC-J Int'l Forwarding Ltd., Co. operates near the NAIA air-cargo complex. MACHip enforces CTPAT compliance through:
 - Section 2.1: Positive two-factor identification at physical warehouse entrance doors.
 - Section 2.2: Immediate digital revocation of credentials for soft-deleted employees.
 - Section 7.3: AES-256 encryption of personnel records and biometric data privacy.

- Section 7.4: Tamper-evident, immutable audit trails capturing user IDs, timestamps, and IP addresses.

6. Comprehensive Academic Defense Reference Sheet

Defense Panel Question	Recommended Technical Response
"Why implement both RFID and Fingerprint biometrics instead of just RFID?"	"An RFID badge represents 'something you have,' which can be shared, lost, or stolen, leading to buddy-punching. By pairing the RFID tap with an optical fingerprint scan ('something you are') within a strict 15-second window, MACHip guarantees physical non-repudiation and enforces CTPAT access security."
"Why are JWT authentication tokens stored in HttpOnly cookies instead of LocalStorage?"	"Tokens stored in LocalStorage are directly accessible to client-side JavaScript, exposing them to token theft via Cross-Site Scripting (XSS). HttpOnly cookies instruct the browser engine to sandbox the token, making it completely invisible to JavaScript while SameSite=Lax prevents CSRF exploits."
"What happens to an employee's attendance and payroll history when an admin deletes their account?"	"MACHip implements Sequelize Paranoid Mode (Soft Deletion). The record is never physically destroyed; it receives a <code>deletedAt</code> timestamp that deactivates login and door access. This permanently preserves historical payroll records and tax logs for mandatory DOLE and CTPAT audits."
"Why deploy on an on-premise Mini PC on the LAN rather than hosting entirely on the public cloud?"	"Logistics air-cargo operations demand 100% door-access and attendance availability. An external internet outage must never lock employees out or halt timekeeping. The LAN Mini PC architecture guarantees full offline operation, exposing only the employee request module via secure port forwarding."
"How does the system ensure raw SQL queries are immune to SQL Injection attacks?"	"All raw database queries in MACHip strictly utilize parameterized replacements with named <code>:param</code> placeholders. User inputs are separated from query syntax and parsed as pure literal data values by PostgreSQL, completely eradicating SQL injection vectors."
"How does the ESP32 communicate with the backend securely over the local Wi-Fi?"	"The hardware layer uses a four-tier defense: (1) a pre-shared API key header, (2) anti-replay timestamp validation with a 120-second threshold, (3) HMAC-SHA256 digital payload signing to detect tampering, and (4) AES-128-CBC payload encryption for sensitive biometric identifiers."
"Why use dual screens (ST7789 Color TFT and SSD1306 OLED) on the hardware terminal?"	"To maintain a clean separation of concerns. The front color TFT provides a clear, professional visual interface for employees (name, photo, time, and status), while the rear OLED provides real-time telemetry (IP address, server latency, and NTP sync) for IT technicians without opening the enclosure."
"Why is the dailyRate in the Payroll table frozen separately from User.dailyRate?"	"To guarantee immutable audit integrity. <code>User.dailyRate</code> is the employee's active, editable salary. When batch payroll is generated, <code>Payroll.dailyRate</code> captures an immutable historical snapshot. If an employee receives a salary raise in November, their January payslips remain accurate and frozen."
"How did you solve the slow loading times on the draft payroll preview?"	"We diagnosed an N+1 query waterfall where the client made 30 sequential HTTP requests for 30 employees (~30 seconds). We engineered <code>/api/payroll/preview-batch</code> , moving the calculation to the backend with concurrent chunking (chunks of 6), reducing response time to 171ms—a 175x speedup."
"What prevents an employee from approving their own leave or overtime requests?"	"MACHip enforces strict Role-Based Access Control (RBAC). Leave and overtime approval endpoints require <code>role_Id = 2</code> (Supervisor) or <code>role_Id = 1</code> (Admin Manager). Additionally, the backend verifies that the approver ID does not match the requester ID."